

Campus Event Manager

Rapport technique — Projet NoSQL Development

Cohorte : MCS DE1 · Master 1 Data Engineering & Cloud Computing

Auteur : ZOUI El Mahdi

Base de données : MongoDB

Stack : Node.js · Express · EJS · Bootstrap 5

Date : 11 septembre 2026

1. Vue d'ensemble du projet

L'application **Campus Event Manager** permet à une université de gérer ses événements de campus (workshops, talks, meetups, hackathons, activités étudiantes) et le suivi des inscriptions étudiantes. L'objectif du projet est de démontrer la conception et l'exploitation d'une base **NoSQL orientée document** avec MongoDB, incluant l'utilisation combinée d'embedding, de références, de mise à jour de tableaux imbriqués et de pipelines d'agrégation.

Stack retenue : Node.js + Express + driver MongoDB officiel + EJS/Bootstrap. Cette stack privilégie la rapidité d'exécution et l'accès direct à la syntaxe MongoDB (les pipelines ne sont pas cachés derrière un ODM).

MongoDB est hébergé sur une VM Ubuntu (nom d'hôte `emzoui`) et joint via la variable d'environnement `MONGODB_URI`. La base est nommée `campus_events`.

2. Modèle de données NoSQL

2.1 Collection `users`

```
{
  "_id": ObjectId,
  "firstName": "Amine",
  "lastName": "Zouiten",
  "email": "amine.zouiten@campus.edu",
  "department": "Data",
  "role": "student",
  "interests": ["nosql", "python", "ml"],
  "createdAt": ISODate
}
```

- `interests` : tableau de strings.
- `email` : indexé unique.
- `role` : énumération contrôlée (*student, teacher, staff, admin*).

2.2 Collection `events`

```
{
  "_id": ObjectId,
  "title": "MongoDB Aggregation Workshop",
  "description": "...",
  "category": "Workshop",
  "tags": ["nosql", "mongodb", "data"],
  "startDate": ISODate,
  "endDate": ISODate,
  "capacity": 25,
  "location": {
    "building": "Innovation Center",
    "room": "B204",
  }
}
```

```

    "campus": "Paris"
  },
  "organizerId": ObjectId,
  "registrations": [
    { "userId": ObjectId, "registeredAt": ISODate, "status": "confirmed" }
  ],
  "createdAt": ISODate
}

```

- location : **objet imbriqué (embedded)**.
- tags : tableau de strings, indexé (multikey).
- registrations : **tableau d'objets embedded**.
- organizerId, registrations.userId : **références** vers users._id.

2.3 Indexes

Index	Type	Justification
users.email	unique	Contrainte métier, recherche par email.
events.startDate	ascending	Tri du catalogue par date, dashboard "prochains events".
events.category	ascending	Filtre côté catalogue et pipeline "par catégorie".
events.tags	multikey	Filtre par tag.
events.registrations.userId	multikey	Recherche des events auxquels un user est inscrit.

2.4 Validation JSON-Schema

Appliquée côté MongoDB (\$jsonSchema dans createCollection) pour garantir :

- title non vide, capacity ≥ 1 , dates typées, structure location complète ;
- role et registrations.status restreints à des valeurs contrôlées ;
- email conforme à un pattern regex.

Une validation applicative complète la validation Mongo (messages d'erreur lisibles en UI), et vérifie notamment que endDate $\geq$ startDate avant chaque insert/update.

3. Choix de modélisation : embedding vs referencing

3.1 Les inscriptions sont *embedded* dans events.registrations

Chaque event a un nombre borné de participants (quelques dizaines à ~200 max), et les inscriptions sont **toujours lues avec l'événement** (page détail, dashboard). Un tableau embedded permet des mises à jour atomiques via \$push/\$pull/\$ positional et un accès à \$size sans join. Une collection séparée aurait multiplié les requêtes sans bénéfice.

3.2 L'organisateur et les utilisateurs sont référencés par ObjectId

Un utilisateur peut être organisateur ou participant de nombreux events ; ses données (email, département...) doivent rester cohérentes en un seul endroit. Dupliquer les infos dans chaque event aurait conduit à une dénormalisation coûteuse à maintenir. Le prix payé — un \$lookup ou

un `find({_id: {$in:[...]}})` — reste faible car les volumes sont raisonnables et les indexes couvrent la jointure.

4. Architecture de l'application

```
src/
├── app.js                # bootstrap Express + EJS + method-override
├── db.js                 # singleton MongoClient (driver v6)
├── routes/              # handlers HTTP par domaine
│   ├── dashboard.js
│   ├── events.js
│   ├── users.js
│   └── analytics.js
├── repositories/        # TOUTES les opérations MongoDB
│   ├── events.js        # CRUD + agrégations + registrations
│   ├── users.js         # CRUD + $lookup + users sans inscription
│   └── analytics.js     # pipelines A..F
├── views/               # templates EJS (5 pages)
└── public/css/style.css
```

La séparation **routes** ↔ **repositories** rend les requêtes et pipelines MongoDB immédiatement identifiables dans le code, comme demandé dans le brief.

5. Pages de l'application

Page	Route	Fonctions clés (côté MongoDB)
Dashboard	/	<code>countDocuments</code> , <code>find + sort + limit</code> , agrégation <code>\$unwind + \$match + \$group</code> pour le total confirmé, <code>\$filter + \$size</code> pour le plus populaire.
Events	/events	CRUD complet (<code>insertOne</code> , <code>updateOne</code> , <code>deleteOne</code>), filtres via <code>\$regex</code> , <code>\$gte/\$lt</code> , tag array, tri, projection.
Détail event	/events/:id	<code>\$push</code> (nouvelle inscription), positional <code>\$</code> (<code>\$set</code> sur <code>registrations.\$.status</code>) pour annuler, <code>\$pull</code> pour retirer, contrôle capacité + anti-doublon.
Users	/users	CRUD, <code>\$lookup</code> avec pipeline pour compter les inscriptions par user, refus de suppression si références.
Analytics	/analytics	6 pipelines d'agrégation (voir §6).

Insérer ici les captures d'écran des 5 pages (une par page) après avoir lancé l'application.

6. Focus sur deux pipelines d'agrégation

6.1 Analyse D — Events au-dessus de l'occupation moyenne

Fichier : `repositories/analytics.js`, fonction `eventsAboveAverageOccupancy`. Ce pipeline illustre le calcul d'une moyenne globale puis un filtrage individuel :

1. `$project` : calcule pour chaque event le nombre d'inscriptions confirmées via `$filter + $size`.
2. `$addFields` : calcule le pourcentage d'occupation `confirmed / capacity × 100` (avec `$cond` pour éviter la division par zéro).
3. `$group _id: null` : agrège tous les events pour calculer la moyenne globale `avg` et pousse chaque document dans un tableau `items`.
4. `$unwind + $replaceRoot + $mergeObjects` : ré-étale les documents et injecte la moyenne globale `overallAverage` dans chacun.
5. `$match: { $expr: { $gt: ["$occupancy", "$overallAverage"] } }` : ne conserve que les events dont l'occupation dépasse la moyenne.
6. `$sort` : tri par occupation décroissante.

Ce pipeline combine 6 stages et démontre l'usage de champs calculés (`$cond`, `$divide`, `$multiply`) et d'un `$expr` pour référencer un champ calculé dans un `$match`.

6.2 Analyse C — Utilisateurs sans inscription

Fichier : `repositories/users.js`, fonction `usersWithNoRegistration`. Ce pipeline utilise un `$lookup` corrélé pour identifier les utilisateurs jamais inscrits :

1. `$lookup` avec `let: { uid: '$_id' }` et un sous-pipeline qui filtre les events dont le tableau `registrations.userId` contient `$$uid` ; `$limit: 1` pour arrêter dès qu'un event est trouvé.
2. `$match: { refs: { $size: 0 } }` : conserve uniquement les utilisateurs pour lesquels le résultat du `$lookup` est vide.
3. `$sort` : tri alphabétique.

L'usage du sous-pipeline dans `$lookup` évite de récupérer l'ensemble des events et permet à MongoDB d'exploiter l'index `events.registrations.userId`.

7. Cohérence et gestion d'erreurs

- Email en doublon → erreur 11000 transformée en HTTP 409.
- Capacité ≤ 0 ou invalide → HTTP 400 avec message clair.
- Date de fin < date de début → HTTP 400.
- Event ou utilisateur introuvable → HTTP 404.
- Inscription en double sur un event → HTTP 409.
- Event complet : l'inscription bascule automatiquement en statut `waiting`.
- Identifiant malformé → HTTP 400 via `ObjectId.isValid`.
- Suppression d'un user encore référencé (organizer ou registration) : bloquée avec message.

8. Conclusion

Difficultés rencontrées

- Trouver le bon équilibre entre embedding (rapidité de lecture, atomicité) et referencing (cohérence) sur les inscriptions.
- Écrire un pipeline lisible pour l'analyse D : injecter une moyenne globale dans chaque document sans passer par deux requêtes séparées.

Limitations

- Pas d'authentification : l'utilisateur "connecté" est simulé via la sélection d'un user dans les formulaires d'inscription.
- Pas de pagination : le volume de données est faible (18 events), inutile pour ce périmètre.

Améliorations possibles

- Ajouter un `changeStream` pour rafraîchir le dashboard en temps réel.
- Précalculer les statistiques du dashboard dans un job planifié + cache Redis pour un service à plus grand volume.
- Ajouter un module d'authentification (Passport) pour restreindre le CRUD aux organisateurs.